

Compiler Design (AKTU) - Top 3 Important Questions per Unit

Unit 1: Introduction to Compilers

1. Phases of a Compiler - Explain each phase with a diagram.
2. Lexical Analysis and Regular Expressions - Role of lexical analyzer; convert regex to NFA/DFA.
3. Symbol Table Management - Operations and implementation of symbol tables.

Unit 2: Syntax Analysis (Parsing)

1. FIRST and FOLLOW Sets - Calculation with examples.
2. LL(1) Parsing Table - Constructing and checking if grammar is LL(1).
3. Elimination of Left Recursion and Left Factoring - Grammar transformation steps.

Unit 3: Semantic Analysis and Intermediate Code

1. Three Address Code (TAC) Generation - For expressions and control flow.
2. Syntax Directed Definitions (SDD) - Inherited and synthesized attributes.
3. Backpatching - For Boolean expressions and control statements.

Unit 4: Code Optimization

1. Basic Blocks and DAG Representation - Constructing basic blocks and DAGs.
2. Code Optimization Techniques - Constant folding, dead code elimination, loop optimizations.
3. Data Flow Analysis - Concepts like reaching definitions and live variable analysis.

Unit 5: Code Generation and Runtime Environments

1. Code Generation Algorithms - Instruction selection, register allocation.
2. Activation Records - Structure and use during procedure calls.
3. Storage Allocation Strategies - Stack vs heap, static vs dynamic memory allocation.

Important Unit-1

Unit 1: Introduction to Compiler

1. What is bootstrapping and cross-compiler?
2. Describe the relationship between finite state machines and regular expressions. Discuss how regular expressions are used in lexical analysis and pattern matching.
3. Given a regular expression $a(b|c)^*$, construct the corresponding Nondeterministic Finite Automaton (NFA) that recognizes the language described by the regular expression
4. Check whether given grammar is ambiguous or not. If ambiguous then convert it into unambiguous grammar: $E \rightarrow E + E | E^* E | id$.
5. Construct the NFA and DFA for the following regular expression.
 $(0+1)^*(00+11)(0+1)^*$
6. Construct predictive parse table for the following grammar.

$E \rightarrow E + T / T$

$T \rightarrow T * F / F$

$F \rightarrow F / a / b$

EduShine
CLASSES
By- ARMAN ALI

Important Unit-2

Unit 2: Basic Parsing Techniques

1. Consider the following grammar for a simple expression language:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id$

Consider expression $id * (id + id)$. Apply shift reduce parsing to construct parse tree.

2. Check whether the given grammar is LR(0) or not: $S \rightarrow PQy$, $P \rightarrow Sy|x$, $Q \rightarrow yS$.
3. Construct the Operator precedence parsing table for the following Grammar.

$E \rightarrow E + T / T$

$T \rightarrow T * F / F$

$F \rightarrow id$

4. Construct the SLR parsing table for the following Grammar.

$S \rightarrow 0S0$

$S \rightarrow 1S1$

$S \rightarrow 10$

5. Construct the CLR parse table for the following Grammar:

$A \rightarrow BB$

$B \rightarrow cB$

$B \rightarrow d$

adushine
CLASSES
By- ARMAN ALI

Important Unit-3

Unit 3: Syntax-directed Translation

1. Differentiate between synthesized and inherited attributes. Write Syntax directed definition of a simple desk calculator.
2. Write syntax directed definition for a given Important statement:

$S \rightarrow id=E$

$E \rightarrow E+E$

$E \rightarrow E * E$

$E \rightarrow -E$

$E \rightarrow (E)$

$E \rightarrow id$

3. Translate the following arithmetic expression into quadruples and triples:

i. $(x + y) * (y + z) + (x + y + z)$

ii. $a = -b * (c + d) + b$

4. What is back patching. Generate three address code for the following Boolean expression using back patching:

$a < b \text{ or } c > d \text{ and } e < f$

5. Write the three-address code for the statement given below:

while $a < b$ **do**

if $c < d$ **then**

$x = y * z$

else

$x = y + z$

Important Unit-4

Unit 4: Symbol Tables & Run-Time Administration

1. Define Symbol table? Explain about the data structures used for symbol table.
2. Discuss the stack allocation and heap allocation strategies of the runtime environment with an example.
3. Draw the format of Activation Record in stack allocation and explain each field in it.
4. What do you understand by lexical phase error and syntactic error? Also suggest methods for recovery of errors.
5. Define semantic errors in a compiler. Discuss the challenges associated with detecting and handling semantic errors. Provide examples to illustrate semantic issues.

EduShine
CLASSES
By- ARMAN ALI

Important Unit-5

Unit 5: Code Generation and Optimization

1. What are basic blocks? Write the algorithm for partitioning into Blocks.
2. Explain in detail about loop optimization.
3. Construct the flow graph for the following code segment:

```
fact(n)
{
  int f=1;
  for(i=2; i≤n; i++)
    f=f*i;
  return f;
}
```

4. Define a DAG. Construct a DAG for the expression:
 $p + p * (q - r) + (q - r) * s$
5. What are the various issues in design of code generator & code loop optimization?
6. Write a short note on:
 - a. Flow graph (with example)
 - b. Dominators
 - c. Natural loops
 - d. Inner loops
 - e. Reducible flow graphs
 - f. Copy Propagation
 - g. Dead-Code Elimination
 - h. Code Motion
 - i. Reduction in Strength.